

17 - Asincronía y Promesas

Asincronía en JavaScript: ¡Como las tareas del día a día!

JavaScript, por naturaleza, es un lenguaje síncrono. Esto significa que, normalmente, ejecuta una tarea a la vez y no pasa a la siguiente hasta que la actual ha terminado. Pero, en la vida real, hay muchas tareas que pueden suceder al mismo tiempo o que no dependen de la finalización de otra. Aquí es donde entra la asincronía: permite que JavaScript haga múltiples cosas sin quedarse "esperando" a que una tarea termine.

¿Qué es la asincronía?

Piensa en la asincronía como en tu rutina diaria. Imagina que pones agua a hervir para el té. Mientras esperas a que el agua esté lista, no te quedas de brazos cruzados, ¿verdad? Tal vez aproveches para revisar correos, preparar un sándwich o simplemente relajarme un rato. ¡Así funciona la asincronía! Estás haciendo varias cosas al mismo tiempo y no estás bloqueado esperando que una tarea termine para comenzar otra.

En el mundo de JavaScript, hay ciertas tareas, como hacer una solicitud a un servidor o leer archivos, que toman tiempo. En lugar de bloquear el código, JavaScript permite que esas tareas "lentas" se ejecuten en segundo plano, mientras el resto del código sigue corriendo.

Ejemplo de vida cotidiana:

Tarea síncrona:

Imagina que haces lo siguiente de manera síncrona:

1. Pones agua a hervir.
2. Te quedas mirando la pava eléctrica sin hacer nada más hasta que el agua hierve.
3. Una vez lista el agua, entonces y solo entonces, preparas el té.
4. Finalmente, te sientas a beber tu té.

Esto sería muy ineficiente, ¿no? ¡Podrías haber hecho otras cosas mientras el agua hervía!

Tarea asíncrona:

Ahora, imagina que haces lo siguiente de manera asíncrona:

- Pones agua a hervir.
- Mientras el agua hierve, preparas la comida, haces una llamada o revisas correos.
- Cuando el agua está lista (y te avisa), terminas de preparar tu té.
- Luego te sientas a disfrutar tu té y comida, sin haber perdido tiempo.

¡Mucho más eficiente! En este escenario, el hervir el agua sería una tarea asíncrona, que corre en el "fondo" mientras tú haces otras cosas.

Asincronía en JavaScript

En JavaScript, hay varias formas de manejar tareas asíncronas. Tradicionalmente, se usaban callbacks (funciones que se ejecutan una vez que una tarea ha terminado), pero con el tiempo evolucionó hacia Promesas y, finalmente, la sintaxis `async/await`.

¡Antes de continuar vamos a profundizar en dos métodos clave que están relacionados con la asincronía en JavaScript: `setTimeout` y `setInterval`!

Ambos métodos son fundamentales para entender cómo manejar tareas que ocurren después de un tiempo o a intervalos regulares, y son muy útiles cuando trabajamos con procesos asincrónicos.

1. `setTimeout`

`setTimeout` nos permite ejecutar una función una vez después de un periodo de tiempo específico. Es una manera sencilla de retrasar la ejecución de una tarea.

Ejemplo de la vida cotidiana:

Piensa en esto como poner un temporizador en el horno. Te preparas para hornear galletas, ajustas el temporizador a 15 minutos y, cuando suena el timbre (después de esos 15 minutos), vas a sacarlas.

En código, funciona así:

```
console.log("Pongo las galletitas en el horno");

setTimeout(() => {
  console.log("Las galletitas están listas");
}, 15000); // 15000 milisegundos = 15 segundos

console.log("Mientras tanto, sigo haciendo otras cosas...");
```

Explicación:

Al ejecutar `setTimeout`, estamos diciendo: "Dentro de 15 segundos, quiero que esta función (en este caso, que me avise que las galletas están listas) se ejecute."

Mientras tanto, JavaScript no se detiene, sigue ejecutando otras líneas de código.

Es importante notar que, aunque escribimos 15000 (milisegundos), el temporizador se mueve en segundo plano, dejando que el resto del código siga funcionando.

Usos comunes:

- Retrasar la ejecución de una tarea.
- Dar tiempo para que otros procesos se completen.
- Crear efectos visuales con retardos, como mostrar un mensaje luego de un tiempo.

2. setInterval

setInterval es similar a setTimeout, pero con una diferencia importante: en lugar de ejecutar la función una sola vez, la ejecuta repetidamente en intervalos regulares.

Ejemplo de la vida cotidiana:

Imagina que estás usando un despertador que te recuerda hacer estiramientos cada 30 minutos. Configuras el temporizador, y cada media hora suena para que te levantes y te estires.

En código, funciona así:

```
setInterval(() => {  
  console.log("¡Hora de estirarse!");  
}, 1800000); // 1800000 milisegundos = 30 minutos
```

Explicación:

Aquí hemos configurado un temporizador que suena cada 30 minutos (o 1.8 millones de milisegundos). Cada vez que suena, muestra el mensaje "¡Hora de estirarse!".

A diferencia de `setTimeout`, que ejecuta la función una vez, `setInterval` lo hace repetidamente.

Deteniendo un `setInterval`:

A veces necesitamos detener el intervalo después de un tiempo. Para ello, usamos `clearInterval`.

```
let intervalo = setInterval(() => {
  console.log("¡Hora de estirarse!");
}, 5000); // cada 5 segundos

setTimeout(() => {
  clearInterval(intervalo); // detener después de 20 segundos
  console.log("Ya no más estiramientos");
}, 20000); // 20000 milisegundos = 20 segundos
```

En este ejemplo, el mensaje se muestra cada 5 segundos, pero después de 20 segundos, detenemos el intervalo usando `clearInterval`.

Resumen:

- `setTimeout` es como un temporizador de cocina que te avisa una vez después de cierto tiempo.
- `setInterval` es como un despertador que suena cada cierto tiempo hasta que lo apagas.

Ambos métodos son muy útiles cuando quieres manejar eventos que ocurren en el futuro, pero no quieres bloquear el flujo normal del código. ¡Y son esenciales para entender cómo funciona la asincronía en JavaScript!

Pensemos en más ejemplos para entender cómo funciona:

Ejemplo 1: Llamada telefónica mientras haces otras tareas

Haces una llamada a un amigo. Mientras esperas a que responda, puedes hacer otras cosas, como revisar tus correos.

```
console.log("Hago una llamada...");
setTimeout(() => {
  console.log("Mi amigo respondió la llamada.");
}, 3000); // Simula una espera de 3 segundos.
console.log("Sigo revisando mis correos mientras espero.");
```

Aquí, la llamada (la tarea asincrónica) no bloquea tu capacidad de seguir haciendo otras cosas (como revisar correos).

Ejemplo 2: Temporizadores y tareas

Si pones una alarma para despertarte después de 5 minutos, no te quedas quieto esperando esos 5 minutos, sigues haciendo otras actividades.

```
console.log("Voy a poner una alarma...");
setTimeout(() => {
  console.log("¡Alarma sonó! Es hora de despertar.");
}, 5000); // 5 segundos simulan 5 minutos.
console.log("Mientras espero, sigo durmiendo...");
```

Ejercicio: Temporizador de Ejercicio Físico

Imagina que estás desarrollando una pequeña aplicación que recuerda a las personas hacer ejercicio después de un cierto tiempo. La aplicación:

1. Avisará a la persona que es hora de empezar su ejercicio después de 5 segundos.
2. Una vez que empiece, cada 2 segundos le recordará que debe continuar moviéndose.
3. Después de 10 segundos de ejercicio, la aplicación parará los recordatorios y dirá que el ejercicio ha terminado.

Código del ejercicio:

```
// Paso 1: Avisar que es hora de empezar el ejercicio después de 5 segundos
setTimeout(() => {
  console.log("¡Es hora de empezar tu ejercicio!");

  // Paso 2: Recordar cada 2 segundos que debe seguir haciendo ejercicio
  let recordatorio = setInterval(() => {
    console.log("¡Sigue moviéndote!");
  }, 2000);

  // Paso 3: Detener el recordatorio después de 10 segundos de ejercicio
  setTimeout(() => {
    clearInterval(recordatorio); // Detener los recordatorios
    console.log("¡Buen trabajo! El ejercicio ha terminado.");
  }, 10000); // 10000 milisegundos = 10 segundos
}, 5000); // 5000 milisegundos = 5 segundos
```

Explicación del ejercicio:

- Primero, usamos `setTimeout` para que después de 5 segundos aparezca el mensaje que indica que es hora de empezar el ejercicio.
- Segundo, dentro de ese mismo `setTimeout`, activamos un `setInterval` que va a mostrar un mensaje cada 2 segundos recordando a la persona que siga haciendo ejercicio.
- Tercero, después de 10 segundos de ejercicio (usando otro `setTimeout`), detenemos el intervalo con `clearInterval` para que los recordatorios paren y mostramos un mensaje final de que el ejercicio ha terminado.

Objetivo del ejercicio:

Aprender a usar `setTimeout` y `setInterval` juntos para manejar eventos asíncronos en diferentes momentos.

Practicar la cancelación de intervalos usando `clearInterval`.

Promesas: "Te lo prometo, te aviso cuando termine"

Una promesa es un objeto que representa el eventual resultado de una operación asincrónica. Cuando haces algo asincrónico (como solicitar datos de un servidor), la promesa "promete" que en algún momento, la operación terminará y obtendrás un resultado: puede ser éxito o puede haber fallado, pero tarde o temprano sabrás qué pasó.

Una promesa puede estar en uno de tres estados:

Pending (Pendiente): La operación aún no ha terminado.

Fulfilled (Cumplida): La operación terminó con éxito.

Rejected (Rechazada): La operación falló.

Ejemplo de la vida real con promesas:

Promesa con tu amigo: Imagina que le pides a tu amigo que te compre una pizza. Tu amigo te "promete" que va a hacerlo. Como mencionamos anteriormente una promesa puede tener varios resultados:

- *Pending:* Tu amigo está en camino a comprar la pizza.
- *Fulfilled:* Tu amigo llega con la pizza, itodo salió bien!
- *Rejected:* Tu amigo no pudo comprar la pizza porque la tienda estaba cerrada.

Mientras esperas a que tu amigo cumpla con la promesa, puedes hacer otras cosas, como poner la mesa o ver la televisión. No necesitas esperar sin hacer nada.

Promesas en JavaScript

Una promesa en código podría verse así:

```
let promesaDePizza = new Promise((resolve, reject) => {
  let hayPizza = true;

  if (hayPizza) {
    resolve("¡La pizza está aquí!");
  } else {
    reject("No hay pizza, la tienda estaba cerrada.");
  }
});

// Luego puedes usar .then() y .catch() para manejar el resultado
promesaDePizza
  .then((mensaje) => {
    console.log(mensaje); // "¡La pizza está aquí!"
  })
  .catch((error) => {
    console.log(error); // "No hay pizza, la tienda estaba
cerrada."
  });
```

Aquí, estamos creando una promesa para simular que alguien va a traernos una pizza. Si hay pizza, la promesa se cumple (resolve) y mostramos un mensaje de éxito. Si no hay pizza, la promesa falla (reject) y mostramos un mensaje de error.

Ejemplo cotidiano: Lavandería

Imagina que envías tu ropa a la lavandería. No te vas a quedar ahí esperando a que terminen, ¿cierto? En lugar de eso, haces otras tareas: ir de compras, leer un libro o ver una película. Cuando la ropa está lista, te avisan y entonces la recoges.

Así es como funcionan las promesas en JavaScript. Pides algo (como los datos de un servidor) y en lugar de bloquear el código mientras esperas, sigues

ejecutando otras cosas. Cuando la respuesta está lista, la promesa te "avisa" y tomas acción.

Ejemplo en código:

```
function enviarALavanderia(ropa) {  
  return new Promise((resolve, reject) => {  
    console.log(`Enviando ${ropa} a la lavandería...`);  
    setTimeout(() => {  
      if (ropa === "limpia") {  
        resolve("Tu ropa está limpia y lista para ser recogida.");  
      } else {  
        reject("¡Hubo un problema con tu ropa!");  
      }  
    }, 3000);  
  });  
}  
  
enviarALavanderia("ropa sucia")  
  .then((mensaje) => console.log(mensaje))  
  .catch((error) => console.log(error));
```

Aquí usamos la función `setTimeout` para simular que la ropa tarda 3 segundos en estar lista. Si la ropa está sucia, el proceso falla. Si todo va bien, obtienes la ropa limpia.

Async/Await: El mejor formato para manejar Promesas

Con `async/await`, el manejo de promesas se vuelve aún más fácil y legible. En lugar de usar `.then()` y `.catch()`, puedes escribir tu código como si fuera síncrono, pero sigue siendo asíncronico.

Ejemplo:

```
async function hacerPedido() {  
  try {  
    let pizza = await pedirPizza();  
    console.log(pizza);  
  } catch (error) {  
    console.log(error);  
  }  
}  
  
hacerPedido();
```

Aquí, `await` espera a que la promesa se resuelva, pero sin bloquear el flujo de tu programa.

Conclusión

La asincronía en JavaScript es esencial para que tus aplicaciones sean rápidas y eficientes. Es como gestionar tu tiempo en la vida diaria: mientras esperas a que algo termine, puedes hacer otras tareas. Las promesas y `async/await` hacen que este proceso sea mucho más fácil y manejable en código.

Recuerda, cuando una operación lleva tiempo, no quieres que tu aplicación "se congele" esperando. Con la asincronía, puedes hacer que todo siga funcionando y, cuando la operación termine, simplemente tomar acción en consecuencia.